Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет прикладної математики та інформатики

Бази даних та інформаційні системи

Лабораторна робота №10

Тема: «Транзакції в СКБД PostgreSQL»

з курсу “Бази даних та інформаційні системи ”

Виконав:

Ст. Яценко Владислав Тарасович

Група: ПМі-33

Львів – 2025

Тема: Вивчення понять транзакції та управління конкурентним доступом в СКБД PostgreSQL.

Мета роботи: Ознайомлення з використанням транзакцій, їх розробкою та застосуванням, рівнями ізоляцій та механізмом управління конкурентним доступом в СКБД PostgreSQL

Хід роботи

Опрацював теоретичні відомості про транзакції, атомарні транзакції, команди, скасування змін, точки збереження, повернення до точок збереження, рівні ізоляції, явні блокування, перевірки цілісності даних, обмеження, блокування та індекси.

ПАРА 1: READ COMMITTED з SAVEPOINT та ROLLBACK TO

Написав транзакцію з демонстрацією рівня ізоляції READ COMMITTED та використанням точок збереження. Транзакція включає збільшення страхових сум полісів з подальшим скасуванням помилкової операції через ROLLBACK TO.

Операції в SESSION 1:

- Збільшення страхових сум полісів на 100,000

```
-- SESSION 1 - КРОК 1: Початок READ COMMITTED з SAVEPOINT
BEGIN;

SELECT POLICY_ID, INSURANCE_AMOUNT FROM INSURANCE_POLICY WHERE POLICY_ID IN (1, 2);
UPDATE INSURANCE_POLICY SET INSURANCE_AMOUNT = INSURANCE_AMOUNT + 100000 WHERE POLICY_ID IN (1, 2);
```

	policy_id [PK] integer	insurance_amount numeric (12,2)
1	1	2150000.00
2	2	2047174.78

	policy_id [PK] integer	insurance_amount numeric (12,2)
1	1	2250000.00
2	2	2147174.78

- Створення SAVEPOINT "after_basic_update"

```
-- Створюємо точку збереження
SAVEPOINT after_basic_update;
SELECT 'SAVEPOINT "after_basic_update" створено' AS checkpoint;
```

	checkpoint text
1	SAVEPOINT "after_basic_update" створено

- Помилкове додавання 4,000,000 до полісу 1

```
-- SESSION 1 - КРОК 2: Помилкова операція та ROLLBACK TO
UPDATE INSURANCE_POLICY SET INSURANCE_AMOUNT = INSURANCE_AMOUNT + 4000000 WHERE POLICY_ID = 1;
SELECT POLICY_ID, INSURANCE_AMOUNT FROM INSURANCE_POLICY WHERE POLICY_ID = 1;
```

	policy_id [PK] integer	insurance_amount numeric (12,2)
1	1	6250000.00

- ROLLBACK TO для скасування помилки

```
-- Відкочуємо помилку
ROLLBACK TO after_basic_update;
SELECT 'ROLLBACK TO "after_basic_update" - помилка скасована' AS rollback_info;
```

	rollback_info text
1	ROLLBACK TO "after_basic_update" - помилка скасована

	policy_id [PK] integer	insurance_amount numeric (12,2)
1	1	2150000.00
2	2	2047174.78

- Правильне додавання 50,000 до полісу 1

	policy_id [PK] integer	insurance_amount numeric (12,2)
1	1	2300000.00

Паралельно в SESSION 2:

- Демонстрація принципу READ COMMITTED
- Читання даних до та після COMMIT в SESSION 1

```
BEGIN;
SELECT 'SESSION 2: READ COMMITTED читач' AS status;
SELECT POLICY_ID, INSURANCE_AMOUNT FROM INSURANCE_POLICY WHERE POLICY_ID IN (1, 2);
```

	status text
1	SESSION 2: READ COMMITTED читач

	policy_id [PK] integer	insurance_amount numeric (12,2)
1	1	2150000.00
2	2	2047174.78

Комітимо зміни в SESSION 1:

```
COMMIT;
SELECT 'КРОК 3A: Транзакція зафіксована!' AS final_step;
```

	final_step text
1	КРОК 3A: Транзакція зафіксована!

Бачимо зміни в SESSION 2:

```
SELECT 'КРОК 3B: Читання після COMMIT в SESSION 1' AS step;
SELECT POLICY_ID, INSURANCE_AMOUNT FROM INSURANCE_POLICY WHERE POLICY_ID IN (1, 2);
```

	policy_id [PK] integer	insurance_amount numeric (12,2)
1	1	2300000.00
2	2	2147174.78

ПАРА 2: DEADLOCK з відновленням через SAVEPOINT
Розпочав першу транзакцію яка буде блокувати поліс 1, створив точку збереження для можливого відновлення.

SESSION 1 - Початок та оновлення
Початкова сума та результат оновлення в SESSION 1

```
UPDATE INSURANCE_POLICY SET INSURANCE_AMOUNT = INSURANCE_AMOUNT + 100000 WHERE POLICY_ID = 1;
SELECT 'Після оновлення (+100000):' AS updated;
SELECT POLICY_ID, INSURANCE_AMOUNT FROM INSURANCE_POLICY WHERE POLICY_ID = 1;
```

	policy_id [PK] integer	insurance_amount numeric (12,2)
1	1	3122537.50

	updated text
1	Після оновлення (+100000):

	policy_id [PK] integer	insurance_amount numeric (12,2)
1	1	3222537.50

SESSION 2 - Читання під час незафіксованих змін

	read_info text
1	SESSION 2 читає поліс 1 (SESSION 1 ще не зробив COMMIT):

	policy_id [PK] integer	insurance_amount numeric (12,2)
1	1	3122537.50

SESSION 2 бачить стару суму (READ COMMITTED принцип)

```
SAVEPOINT before_our_changes;
SELECT 'SAVEPOINT створено' AS savepoint_created;
```

	savepoint_created text
1	SAVEPOINT створено

COMMIT в SESSION 1

```
COMMIT;
SELECT 'SESSION 1: Зміни зафіксовані!' AS committed;
```

	committed text
1	SESSION 1: Зміни зафіксовані!

SESSION 2 - Тепер бачимо зміни

```
SELECT 'Читаємо після COMMIT SESSION 1:' AS after_commit;
SELECT POLICY_ID, INSURANCE_AMOUNT FROM INSURANCE_POLICY WHERE POLICY_ID = 1;
```

	policy_id [PK] integer	insurance_amount numeric (12,2)
1	1	3222537.50

```
SELECT 'Додаємо занадто багато:' AS risky;
UPDATE INSURANCE_POLICY SET INSURANCE_AMOUNT = INSURANCE_AMOUNT + 500000 WHERE POLICY_ID = 1;
SELECT POLICY_ID, INSURANCE_AMOUNT FROM INSURANCE_POLICY WHERE POLICY_ID = 1;
```

	risky text
1	Додаємо занадто багато:

	policy_id [PK] integer	insurance_amount numeric (12,2)
1	1	3722537.50

SESSION 2 бачить нові дані + ризикове оновлення

SESSION 2 - ROLLBACK TO та правильне оновлення

ROLLBACK TO + правильне оновлення + COMMIT

```
SELECT 'Занадто багато - скасовуємо:' AS rollback_reason;
ROLLBACK TO before_our_changes;
```

	rollback_reason text
1	Занадто багато - скасовуємо:

```
ROLLBACK

Query returned successfully in 42 msec.
```

```
SELECT 'Після ROLLBACK TO:' AS after_rollback;
SELECT POLICY_ID, INSURANCE_AMOUNT FROM INSURANCE_POLICY WHERE POLICY_ID = 1;
```

	after_rollback text
1	Після ROLLBACK TO:

	policy_id [PK] integer	insurance_amount numeric (12,2)
1	1	3222537.50

	moderate text
1	Помірне оновлення:

```
UPDATE 1

Query returned successfully in 43 msec.
```

```
COMMIT;
SELECT 'SESSION 2 завершено!' AS done;
```

	done text
1	SESSION 2 завершено!

```
SELECT 'ФІНАЛЬНИЙ РЕЗУЛЬТАТ:' AS final;
SELECT
    POLICY_ID,
    INSURANCE_AMOUNT,
    'SESSION 1: +100000, SESSION 2: +50000 = ЗАГАЛОМ +150000' AS summary
FROM INSURANCE_POLICY
WHERE POLICY_ID = 1;
```

	final text
1	ФІНАЛЬНИЙ РЕЗУЛЬТАТ:

	policy_id [PK] integer	insurance_amount numeric (12,2)	summary text
1	1	3372537.50	SESSION 1: +100000, SESSION 2: +50000 = ЗАГАЛОМ +150000

ВІДПОВІДІ НА ПИТАННЯ ПРО ТРАНЗАКЦІЇ

1. Поясніть у чому суть поняття транзакції

Транзакція - це послідовність операцій з базою даних, яка виконується як одна неподільна одиниця роботи за принципом "все або нічого". Якщо хоча б одна операція не виконається, всі зміни скасовуються.

2. Назвіть властивості транзакцій

Atomicity (Атомарність)
Consistency (Узгодженість)
Isolation (Ізоляція)
Durability (Довговічність)

3. Що означає атомарність транзакції?

Атомарність гарантує, що транзакція виконується повністю або не виконується зовсім. Неможливо виконати лише частину операцій транзакції - або всі операції фіксуються, або всі скасовуються.

4. Що означає узгодженість транзакції?

Узгодженість забезпечує, що транзакція переводить базу даних з одного коректного стану в інший коректний стан, дотримуючись всіх обмежень цілісності та бізнес-правил.

5. Що означає ізоляція транзакції?

Ізоляція визначає, наскільки зміни однієї транзакції видимі для інших паралельних транзакцій. Це захищає від взаємного впливу одночасно виконуваних транзакцій.

6. Що означає довговічність транзакції?

Довговічність гарантує, що після успішного завершення (COMMIT) транзакції, всі зміни залишаються в базі даних назавжди, навіть у випадку збоїв системи.

7. Що таке точка збереження транзакції?

Точка збереження (SAVEPOINT) - це іменована мітка всередині транзакції, до якої можна повернутися за допомогою ROLLBACK TO, скасувавши лише частину операцій транзакції.

8. Що таке відкат виконання транзакції?

Відкат (ROLLBACK) - це скасування всіх або частини змін транзакції. ROLLBACK скасовує всю транзакцію, ROLLBACK TO - скасовує зміни до конкретної точки збереження.

9. Які є рівні ізоляції транзакцій?

READ UNCOMMITTED - найнижчий рівень
READ COMMITTED - за замовчуванням в PostgreSQL
REPEATABLE READ - середній рівень
SERIALIZABLE - найвищий рівень

10. На основі якого механізму реалізовано рівні ізоляції транзакцій?

В PostgreSQL використовується механізм MVCC (Multi-Version Concurrency Control) - багатоверсійний контроль паралельності, який створює різні версії записів для різних транзакцій.

11. Що означає поняття «брудне читання»?

Брудне читання - це коли транзакція читає незафіксовані зміни іншої транзакції. Ці дані можуть бути скасовані, тому вони "брудні" (ненадійні).

12. Що означає поняття «неповторюване читання»?

Неповторюване читання - це коли транзакція читає той самий запис двічі і отримує різні значення через зміни, зроблені іншою транзакцією між читаннями.

13. Що означає поняття «фантомне читання»?
Фантомне читання - це коли транзакція виконує той самий запит двічі і отримує різну кількість записів через додавання/видалення записів іншою транзакцією.

14. Що означає поняття «аномалія серіалізації»?
Аномалія серіалізації - це ситуація, коли результат паралельного виконання транзакцій відрізняється від будь-якого можливого послідовного виконання цих же транзакцій, що порушує принцип серіалізації.

Висновок: на цій лабораторній роботі я ознайомився з поняттям транзакцій, команд, скасування змін, рівнів ізоляції, перевірки цілісності даних.